arXiv:2610.03098v1 [cs.AI] 2 Oct 2026

Predictor-Guided Latent Space Codon Optimization for Maximizing Protein Expression

Alberto Caron[1] **Tianyu Cui**[1] **Dmytro S. Lituiev**[1] **Mangal Prakash**[2]
Artem Moskalev[3] **Amina Mollaysa**[1] **Bo Zhai**[1] **Hirsh Nanda**[1]
Daniel M. Poole[1] **Zhongyin Liu**[1] **Iman Farasat**[1] **Robert Davidson**[1]
Nikolay V. Manyakov[1] **Tommaso Mansi**[1] **Scott Oloff**[1] **Rui Liao**[1]
[1]Johnson & Johnson Innovative Medicine
[2]Ellison Institute of Technology Oxford
[3]New Theory AI

Abstract

Codon optimization, the process of selecting synonymous codons to improve mRNA translation efficiency and protein expression, is central to therapeutic protein production and mRNA vaccines, yet it remains a hard problem. The design space is discrete and combinatorially large, precluding gradient-based methods, and existing tools rely on heuristic proxies (e.g., Codon Adaptation Index or GC-content) that poorly capture true expression. We introduce Latent-Space Codon Optimization (LSCO), which recasts this discrete problem as a continuous one by mapping sequences into the latent space of a pretrained mRNA language model, enabling efficient gradient-based search. LSCO combines four components: a data-driven expression objective from an uncertainty-aware predictor, a Minimum-Free-Energy regularizer for structural stability, a naturalness prior from a protein-to-codon back-translation model, and constrained decoding for protein fidelity. On a real-world, wet-lab antibody expression dataset, LSCO outperforms simple frequency-based, as well as modern deep generative baselines in predicted expression, while retaining suitable biophysical properties.

1 Introduction

Synonymous codons, i.e., nucleotide triplets that redundantly encode the same amino acid, create a vast design space for tailoring mRNA without changing the associated protein [7]. This space grows combinatorially with protein length: a therapeutic antibody such as trastuzumab (made of a 449 and 214 amino acid heavy and light chains, respectively) admits on the order of 10^{300} synonymous encodings [8]. Codon choice within this space strongly affects translation efficiency and protein yield, so codon optimization is a key lever in applications from monoclonal antibody (mAb) manufacturing to mRNA therapeutics [29, 33].

Codon optimization faces two challenges. First, the synonymous space is discrete and combinatorially vast, ruling out exhaustive search and gradient-based methods. Second, the true objective, an accurate mRNA-to-protein expression model, is inaccessible: the biology is only partly understood and labelled data is scarce, forcing reliance on heuristic proxies (CAI, GC-content, codon-pair bias, "naturalness") that enable tractable algorithms, but often fail to capture true protein expression [36, 9]. Note that, while the optimization objective is formulated in terms of protein expression, experimental validation typically uses A280 absorbance as a practical proxy for protein yield and abundance [32]. To address both challenges, we propose Latent-Space Codon Optimization (LSCO), which recasts codon optimization as a continuous, gradient-based problem grounded in a data-driven expression

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: AI for Drug Discovery: Bridging the Translation Gap. Corresponding authors: acaron3@its.jnj.com, tcui8@its.jnj.com, dlituiev@its.jnj.com.

objective. LSCO optimizes a continuous latent representation derived from a pretrained mRNA language model, bypassing discrete search, and integrates four components: i) an expression objective from an uncertainty-aware predictor trained on mRNA–protein data that penalizes out-of-distribution drift during optimization; ii) a Minimum Free Energy (MFE) regularizer that avoids near-adversarial, unstable sequences; iii) a naturalness prior from a protein-to-codon back-translation model, learned from the Observed Antibody Space (OAS) data [31], that keeps candidates within plausible codon usages; iv) constrained decoding that ensures protein fidelity.

Our contributions can be summarized as follows:

- We propose LSCO, a continuous latent-space codon optimizer featuring the four novel components listed above that replaces heuristic proxies with reliable expression surrogate as main objective.
- On a real-world, wet-lab antibody expression dataset, LSCO outperforms frequency-based and modern deep generative baselines on predicted expression, as confirmed by independent held-out predictors, while retaining structural stability and host-appropriate GC content.
- Ablations show the MFE regularizer offers a controllable expression–stability trade-off, and both the pretrained latent space (vs. a direct input-space search) and the naturalness prior drive the gains.

2 Background and Related Work

Problem Setup. A protein sequence of length L is denoted by $\mathbf{a} = (a_1, \ldots, a_L)$, where $a_i \in \mathcal{A}$ are the 20 canonical amino acids. Each amino acid a_i is encoded by many synonymous mRNA sequences $\mathbf{c} = (c_1, \ldots, c_L)$, with $c_i \in \mathcal{C}(a_i)$ being the synonymous codons for a_i. We represent codons in one-hot form $R \in [0,1]^{L \times 64}$, where $\mathbf{r}_i$ is the one-hot vector encoding of codon c_i. Codons can be viewed as drawn from a $c_i \sim \text{Categorical}(\tilde{\mathbf{r}}_i)$ distribution, with probabilities $\tilde{\mathbf{r}}_i$. Codon optimization searches the combinatorial design space of codons mapping to the same protein, $\mathcal{S}(\mathbf{a}) = \mathcal{C}(a_1) \times \cdots \times \mathcal{C}(a_L)$, for an mRNA sequence $\mathbf{c}^\star$ that maximizes downstream protein expression,

$$\mathbf{c}^\star = \arg\max_{\mathbf{c} \in \mathcal{S}(\mathbf{a})} \Phi(\mathbf{c}), \tag{1}$$

where Φ is a surrogate mRNA-to-protein expression model. Both the inaccessibility of Φ (underlying biology is only partly understood and labelled data is scarce) and the discrete, combinatorial nature of $\mathcal{S}(\mathbf{a})$ make this problem intractable for exhaustive or gradient-based search [30, 3].

Related Work. Codon optimization has evolved from frequency-based tools (JCat [16], OPTIMIZER [35]) that maximize CAI and codon-pair bias but neglect determinants such as mRNA stability [48, 33], to deep generative mRNA language models pre-trained to learn codon usage from large scale datasets without optimizing any sequence properties post-hoc (CodonBERT [25], CodonTransformer [13], GEMORNA [47]) and per se multi-objective post-hoc sequence optimizers (RNop [15], ICOR [20]). Despite advances in sequence modeling and optimization, these methods generally optimize surrogate sequence features rather than protein expression, creating a potential gap between optimization objectives and the desired biological outcome.

General-purpose methods for optimizing discrete sequences (not just codon optimization) differ in *search space geometry* (*where* they search) and *trust mechanism* (*how* they avoid "reward hacking", i.e., exploiting an unreliable surrogate off-distribution). *Sequence-space* methods operate directly on sequences or distributions over them [10, 5, 40, 2], and evolve candidates by mutation, selection and screening [10, 44, 46]; whereas *latent-space* methods operate in a learned continuous space shared by a decoder and a property prediction model [14, 41], with latent-space Bayesian optimization adding a probabilistic surrogate for uncertainty-aware search [37, 17, 1]. We include an extended version of related work in Appendix A.

LSCO belongs to the *latent-space* family and introduces three novel components: (i) it enforces realistic codon-usage priors using a mRNA foundation autoencoder model pretrained on a large scale dataset; (ii) it constructs a meaningful expression surrogate by leveraging a predictor trained on a domain-specific dataset, instead of relying on heuristic proxies, and embeds this within a multi-criteria optimization objective that further balances stability, naturalness and protein fidelity; (iii) it penalizes out-of-distribution drift during optimization via epistemic uncertainty estimates obtained from a deep ensemble version of the expression surrogate predictive model.

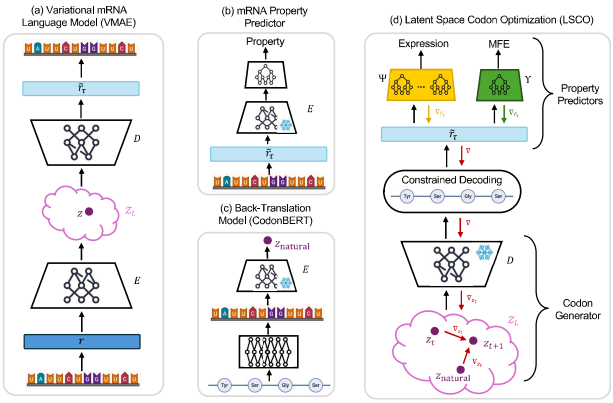


Figure 1: **LSCO framework.** **(a)** mRNA sequences are mapped to codon-latent space using a pretrained variational masked language model. **(b)** mRNA property predictor reuses the pretrained encoder in (a) and takes the soft and continuous version of the discrete sequence. **(c)** We generate a natural mRNA sequence using a pretrained protein-to-codon back-translation model, whose representation $\mathbf{z}_{\text{nat}}$ from the pretrained encoder in (a) is used as the naturalness prior. **(d)** Latent representations $\mathbf{z}$ are optimized with respect to a data-driven expression predictor Ψ, an MFE predictor MFE, and a naturalness prior $\mathbf{z}_{\text{nat}}$ from (c). Constrained decoding ensures valid codon choices, preserving the target protein sequence. The result is an optimized mRNA that balances expression and stability.

3 Latent-Space Codon Optimization

We propose Latent-Space Codon Optimization (LSCO) to address the challenges of codon optimization. First, LSCO recasts the discrete search into a continuous one: rather than searching the mRNA sequences space directly, we optimize codon-level embeddings in the latent space of a pretrained mRNA language model, then decode them back to the discrete sequence space. Second, instead of relying on heuristic proxies such as CAI or naturalness, we train a model $\Psi : [0,1]^{L\times 64} \to \mathbb{R}$ to predict mRNA-to-protein yield, and use a utility over its predictions, U_Ψ–an uncertainty-aware, saturating transform of Ψ–as main objective. Since optimizing expression alone can yield near-adversarial sequences with poor stability or GC content, we add two regularizers: a predicted Minimum Free Energy term, MFE : $[0,1]^{L\times 64} \to \mathbb{R}$, and a naturalness prior U_{nat}, together producing sequences that are highly expressed, stable, and within a host-appropriate GC range. The overall LSCO framework is depicted in Figure 1. Given a target protein $\mathbf{a}$, the LSCO objective is

$$\hat{\mathbf{z}} = \arg\max_{\mathbf{z}} \Big\langle \lambda_\Psi \cdot U_\Psi(D(\mathbf{z})) - \lambda_{\text{MFE}} \cdot \text{MFE}(D(\mathbf{z})) + \lambda_{\text{nat}} \cdot U_{\text{nat}}(\mathbf{z}) \Big\rangle \quad s.t. \quad D(\mathbf{z}) \in \mathcal{S}(\mathbf{a}), \quad (2)$$

where positive $\lambda_\Psi, \lambda_{\text{MFE}}, \lambda_{\text{nat}}$ re-weight expression, MFE, and the naturalness prior; D is the latent-to-sequence decoder (generator); and $\mathcal{S}(\mathbf{a})$ is the synonymous codons space for given protein $\mathbf{a}$. LSCO builds on four main components (Figure 1; full details in Appendix B):

Appendix B).

i) Variational mRNA language model: a bidirectional transformer we pretrain on the curated OAS-mRNA dataset [45] (15M+ sequences) with a Variational Masked Auto-Encoder objective [19]. Its encoder $E : \mathbb{R}^{L\times 64} \to \mathcal{Z}_L$ maps codons to a continuous latent space $\mathcal{Z}_L$, while its decoder $D : \mathcal{Z}_L \to \mathbb{R}^{L\times 64}$ reconstructs codons through a temperature-τ softmax, that can yield a continuous relaxation $\tilde{\mathbf{r}}_\tau$ of the codon sequence that is a smooth, differentiable surrogate for the discrete mRNA.

ii) Expression predictor: a deep learning model $\Psi : [0,1]^{L\times 64} \to \mathbb{R}$, built on top of the pretrained encoder E and trained on mRNA–protein expression data, that takes $\tilde{\mathbf{r}}_\tau$ as inputs rather than $\mathcal{Z}_L$ (which keeps prediction tied to the sequence and enables constrained decoding). We implement Ψ as a deep ensemble [24] of M=10 members (Appendix C.5), which outputs an ensemble mean μ_Ψ and standard deviation σ_Ψ. The expression-related optimization utility in Eq. 2 is a Lower Confidence Bound (LCB) constructed on these predictions

$$U_\Psi(\tilde{\mathbf{r}}_\tau) = \mu_\Psi(\tilde{\mathbf{r}}_\tau) - \lambda_{\text{LCB}}\, \sigma_\Psi(\tilde{\mathbf{r}}_\tau), \quad (3)$$

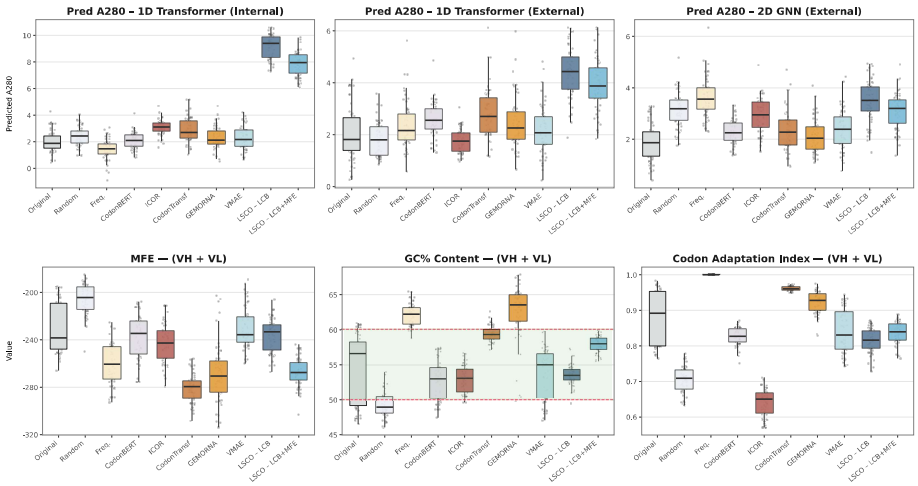


Figure 2: **Optimized sequence comparison on 54 mAbs.** *Top row:* predicted A280 from three models: LSCO's native 1D transformer, an external 1D transformer, and an external structure-aware 2D GNN. LSCO variants attain the highest or comparable expression across all three. *Bottom row:* biophysical metrics on heavy + light chains: MFE (lower = more stable), GC% (green band = host-appropriate range), CAI. *LSCO-LCB+MFE* variant recovers stability and nudges GC/CAI upward at a modest expression cost, staying within host-appropriate ranges.

which maximizes protein yield while penalizing high-uncertainty, out-of-distribution designs.

iii) Minimum Free Energy (MFE) regularizer: since optimizing expression alone in Eq. 2 might yield unstable sequences, we add a regularizing terms based on a predictor MFE : $[0,1]^{L\times 64} \to \mathbb{R}$ (similar architecture as Ψ, no ensemble), trained on ViennaRNA-computed MFE labels [27]; lower MFE implies more stable structure [18].

iv) Naturalness prior: We pretrain a CodonBERT back-translation model [12, 25] on the OAS-mRNA data. This model learns a natural conditional codon distribution $p(\mathbf{c}|\mathbf{a}) = \prod_i p(c_i|\mathbf{a})$ and produces a plausible reference mRNA sequence $\mathbf{c}_{\text{nat}} = \arg\max_{\mathbf{c}} p(\mathbf{c}|\mathbf{a})$. We add a naturalness-based regularizer that penalizes sequences far from $\mathbf{z}_{\text{nat}}$ in the latent-space to keep them within biologically plausible codon usage $U_{\text{nat}}(\mathbf{z}) = -\|E(\mathbf{c}) - E(\mathbf{c}_{\text{nat}})\|_2 = -\|\mathbf{z} - \mathbf{z}_{\text{nat}}\|_2$.

Finally, **constrained decoding** is implemented to guarantee protein fidelity: a per-position validity mask sets non-synonymous codon logits to $-\infty$ before applying temperature-τ softmax, in order to enforce $\tilde{\mathbf{r}}_\tau \in \mathcal{S}(\mathbf{a})$ while the optimizer explores synonymous codon choices. More details about LSCO implementation and a LSCO pseudocode (Algorithm 1) are provided in Appendix B.

4 Experiments

We benchmark LSCO against other methods from three different families: i) frequency-based, host-preferred selection [36]; ii) a maximum-naturalness CodonBERT back-translation model [25]; and iii) modern deep generative models/optimizers, including ICOR [20], CodonTransformer [13], and GEMORNA [47]. Random synonymous codon substitution and the Variational Masked Auto-Encoder (VMAE) generator (without optimizing for predicted expression) are included to serve as lower-bound references. We assess generated mRNA sequences using four criteria: *protein yield*, measured via predicted absorbance at 280 nm (A280) from our native 1D predictor Ψ and two independent held-out 1D and 2D predictors; *thermodynamic stability* via Minimum Free Energy (MFE) computed via the ViennaRNA package [27]); *host compatibility* via GC content (GC%) [23]; and *codon adaptation* via Codon Adaptation Index (CAI), reported as a conventional, but coarse, proxy for expression.

LSCO Evaluation. We evaluate LSCO and baselines on 54 starting monoclonal antibodies (mAbs), selected based on their potential margin of improvement and stratified by target antigen (full details on the selection in Appendix B). We score every optimized design by predicted A280 with the native

Study	Space	Configuration			Predicted A280			Metrics (VH + VL)		
		λ_{MFE}	λ_{LCB}	λ_{nat}	1D Internal	1D External	2D External	MFE	GC%	CAI
MFE Sensitivity	LSCO	0	0	0.01	$\mathbf{9.585 \pm 0.218}$	$\mathbf{4.518 \pm 0.223}$	$\mathbf{3.492 \pm 0.203}$	-235.187 ± 3.555	53.303 ± 0.423	0.816 ± 0.008
		0	1	0.01	$\mathbf{9.156 \pm 0.241}$	$\mathbf{4.400 \pm 0.254}$	$\mathbf{3.485 \pm 0.212}$	-236.711 ± 3.603	53.601 ± 0.390	0.815 ± 0.009
		0.1	0	0.01	8.180 ± 0.237	3.900 ± 0.224	3.175 ± 0.217	-266.274 ± 2.783	57.756 ± 0.285	0.830 ± 0.009
		0.1	1	0.01	7.858 ± 0.235	3.935 ± 0.265	3.099 ± 0.187	$\mathbf{-267.050 \pm 3.211}$	57.916 ± 0.300	0.837 ± 0.008
		1	0	0.01	8.065 ± 0.232	3.887 ± 0.200	3.319 ± 0.201	-263.765 ± 3.122	58.003 ± 0.304	0.825 ± 0.009
		1	1	0.01	7.899 ± 0.256	3.951 ± 0.271	2.923 ± 0.182	$\mathbf{-266.365 \pm 3.381}$	58.018 ± 0.317	$\mathbf{0.838 \pm 0.008}$
Naturalness Ablation	LSCO	0	1	0	9.024 ± 0.247	$\mathbf{4.564 \pm 0.240}$	3.248 ± 0.193	-236.491 ± 3.900	53.428 ± 0.465	0.808 ± 0.008
		0.1	1	0	7.956 ± 0.208	3.885 ± 0.249	3.259 ± 0.254	-265.006 ± 3.066	57.860 ± 0.305	0.836 ± 0.009
Search-Space Ablation	DISO	0	1	0.01	8.547 ± 0.314	4.423 ± 0.240	3.220 ± 0.192	-239.578 ± 3.904	53.594 ± 0.461	0.829 ± 0.007
		0	1	0	8.528 ± 0.305	4.393 ± 0.230	3.036 ± 0.174	-239.561 ± 3.678	53.688 ± 0.421	0.829 ± 0.007
		0.1	1	0.01	7.631 ± 0.268	3.828 ± 0.241	2.939 ± 0.206	-264.987 ± 3.625	57.428 ± 0.353	0.839 ± 0.010
		0.1	1	0	7.598 ± 0.246	3.840 ± 0.247	2.923 ± 0.187	-264.994 ± 3.639	57.382 ± 0.355	$\mathbf{0.840 \pm 0.010}$

Table 1: **Sensitivity and ablation studies.** Predicted protein yield (A280) from the internal 1D Transformer and two held-out evaluators, together with biophysical metrics for the joint heavy and light chains, on the same 54 starting mAbs as Figure 2. Values are means with 95% confidence bands. LSCO searches in the pretrained VMAE latent space, whereas DISO optimizes codon logits directly in input space. The λ_{MFE} and λ_{LCB} columns respectively control MFE regularization and the uncertainty penalty; $\lambda_{\mathrm{nat}} = 0$ denotes removal of the naturalness prior. Overall, $\lambda_{\mathrm{MFE}} = 0.1$ provides good expression–stability trade-off, and LSCO generally outperforms DISO.

1D predictor and the two *independent* held-out evaluators, an external 1D transformer with a different backbone and a structure-aware 2D GNN. We test two LSCO variants: *LSCO–LCB*, optimizing only expression U_{Ψ} and naturalness U_{nat} (with $\lambda_{\mathrm{MFE}} = 0$), and *LSCO–LCB+MFE*, which adds MFE regularization ($\lambda_{\mathrm{MFE}} = 0.1$ unless otherwise specified).

As shown in the top row of Figure 2, LSCO attains the highest predicted A280 across both 1D internal and external predictors and scores comparably on the 2D GNN evaluator. The general agreement among the predictors, despite their different inductive biases, indicates the gains are not just an artifact of the optimizer exploiting its own surrogate, but reflect genuinely higher predicted expression as a direct benefit of optimizing expression in a continuous latent space rather than proxy surrogates. The bottom row shows the stability/host-compatibility trade-off. The expression-only *LSCO–LCB* variant maximizes A280 but delivers weaker MFE than strong baselines like CodonTransformer and GEMORNA. Adding the MFE regularizer (*LSCO–LCB+MFE*) recovers stability by driving MFE substantially lower and nudging GC and CAI upward, yet well within host-appropriate ranges, at a modest expression cost.

Ablation and Sensitivity Studies. We run two additional experiments on LSCO components: a sensitivity analysis sweeping over different MFE weights, and an ablation study aimed at isolating the effect of pretrained VMAE latent space search vs. direct input-space search. Both report predicted A280 from the three predictors plus MFE, GC%, and CAI.

The *MFE Sensitivity* block in Table 1 sweeps λ_{MFE} regularization weight while toggling λ_{LCB}, which sets the uncertainty penalty strength in Eq. 3. The expression-only variant ($\lambda_{\mathrm{MFE}} = 0$) gives the highest A280 across all predictors but the weakest stability and lowest GC. A light penalty ($\lambda_{\mathrm{MFE}} = 0.1$) drives MFE substantially lower and lifts GC and CAI into the host-appropriate range at a modest expression cost. Larger weights ($\lambda_{\mathrm{MFE}} = 1$) yield no further stability. Enabling the uncertainty penalty ($\lambda_{\mathrm{LCB}} = 1$) only slightly lowers A280 while keeping the optimization from drifting off-distribution. Thus, we set $\lambda_{\mathrm{MFE}} = 0.1, \lambda_{\mathrm{LCB}} = 1$ for *LSCO–LCB+MFE* (Figure 2).

The *Search-Space Ablation* block of Table 1 compares LSCO against Direct Input-Space Optimization (DISO), which optimizes per-position codon logits directly, with no VMAE decoder in the loop, sharing all else with LSCO (details in Appendix D). LSCO generally exceeds DISO on predicted A280 across evaluators and on biophysical metrics, showing that optimizing in the pretrained latent space is beneficial. Ablating the naturalness prior ($\lambda_{\mathrm{nat}} = 0$), corresponding to the *Naturalness Ablation* block of Table 1, leaves the native predictor's A280 largely unchanged but degrades the held-out 2D-GNN score—its expected signature as an on-manifold regularizer that keeps designs plausible to independent evaluators rather than inflating the optimizer's own objective.

5 Conclusion

We introduced Latent-Space Codon Optimization (LSCO), which recasts codon optimization as a continuous, data-driven search in the latent space of a pretrained mRNA language model, steered by an uncertainty-aware expression predictor, MFE regularization, a naturalness prior, and constrained decoding to guarantee protein fidelity. On antibody data, LSCO outperforms frequency-based and modern deep generative methods (ICOR, CodonTransformer, GEMORNA) on predicted expression from internal and two external models. Ablation and sensitivity studies demonstrate the benefits of balancing translation efficiency (A280) and stability (MFE) and operating in the pretrained VMAE latent space. As reliable mRNA-protein expression data are scarce, our evaluation relies on predicted rather than empirical yield. Future work will include wet-lab validation and design of a lab-in-the-loop Bayesian optimization framework.

References

[1] A. N. Amin, N. Gruver, Y. Kuang, L. Li, H. Elliott, C. McCarter, A. Raghu, P. Greenside, and A. G. Wilson. Bayesian optimization of antibodies informed by a generative model of evolving sequences. *arXiv preprint arXiv:2412.07763*, 2024.

[2] C. Angermueller, D. Dohan, D. Belanger, R. Deshpande, K. Murphy, and L. Colwell. Model-based reinforcement learning for biological sequence design. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] C. Arbib, M. Ç. Pınar, F. Rossi, and A. Tessitore. Codon optimization by 0-1 linear programming. *Computers & Operations Research*, 119:104932, 2020.

[4] C. Benjamins, S. Surana, O. Bent, M. Lindauer, and P. Duckworth. Bayesian optimization for protein sequence design: Back to simplicity with gaussian processes. In *NeurIPS 2024 AI4MAT Workshop*, 2024.

[5] D. H. Brookes and J. Listgarten. Design by adaptive sampling. In *NeurIPS Workshop on Machine Learning for Molecules and Materials*, 2018.

[6] D. H. Brookes, H. Park, and J. Listgarten. Conditioning by adaptive sampling for robust design. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 773–782. PMLR, 2019.

[7] C. E. Brule and E. J. Grayhack. Synonymous codons: choose wisely for expression. *Trends in Genetics*, 33(4):283–297, 2017.

[8] P. Carter, L. Presta, C. M. Gorman, J. B. B. Ridgway, D. Henner, W. L. T. Wong, A. M. Rowland, C. Kotts, M. E. Carver, and H. M. Shepard. Humanization of an anti-p185^{HER2} antibody for human cancer therapy. *Proceedings of the National Academy of Sciences of the USA*, 89(10):4285–4289, 1992.

[9] D. A. Constant, J. M. Gutierrez, A. V. Sastry, R. Viazzo, N. R. Smith, J. Hossain, D. A. Spencer, H. Carter, A. B. Ventura, M. T. Louie, et al. Deep learning-based codon optimization with large-scale synonymous variant datasets enables generalized tunable protein expression. *bioRxiv*, pages 2023–02, 2023.

[10] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.

[11] E. A. Demissie, S.-Y. Park, J. H. Moon, and D.-Y. Lee. Comparative analysis of codon optimization tools: Advancing toward a multi-criteria framework for synthetic gene design. *Journal of Microbiology and Biotechnology*, 35:e2411066, 2025.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding, 2019.

[13] A. Fallahpour, V. Gureghian, G. J. Filion, A. B. Lindner, and A. Pandi. CodonTransformer: a multispecies codon optimizer using context-aware neural networks. *Nature Communications*, 16(1):3205, 2025.

[14] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, D. Sheberla, J. Aguilera-Iparraguirre, T. D. Hirzel, R. P. Adams, and A. Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS central science*, 4(2):268–276, 2018.

[15] Z. Gong, Z. Jiang, W. Gao, D. Zhuo, and L. Ma. A new deep-learning-based approach for mRNA optimization: High fidelity, computation efficiency, and multiple optimization factors. *arXiv preprint arXiv:2505.23862*, 2025.

[16] A. Grote, K. Hiller, M. Scheer, R. Münch, B. Nörtemann, D. C. Hempel, and D. Jahn. JCat: a novel tool to adapt codon usage of a target gene to its potential expression host. *Nucleic acids research*, 33(suppl_2):W526–W531, 2005.

[17] N. Gruver, S. D. Stanton, N. C. Frey, T. G. J. Rudner, I. Hotzel, J. Lafrance-Vanasse, A. Rajpal, K. Cho, and A. G. Wilson. Protein design with guided discrete diffusion. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[18] P. G. Higgs. RNA secondary structure: physical and computational aspects. *Quarterly reviews of biophysics*, 33(3):199–253, 2000.

[19] D. Hu, X. Hou, X. Du, M. Zhou, L. Jiang, Y. Mo, and X. Shi. Varmae: Pre-training of variational masked autoencoder for domain-adaptive language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6276–6286, 2022.

[20] R. Jain, A. Jain, E. Mauro, K. LeShane, and D. Densmore. ICOR: improving codon optimization with recurrent neural networks. *BMC bioinformatics*, 24(1):132, 2023.

[21] W. Jin, J. Wohlwend, R. Barzilay, and T. Jaakkola. Iterative refinement graph neural network for antibody sequence-structure co-design. *arXiv preprint arXiv:2110.04624*, 2021.

[22] M. Kim, J. Gu, Y. Yuan, T. Yun, Z. Liu, Y. Bengio, and C. Chen. Offline model-based optimization: Comprehensive review. *Transactions on Machine Learning Research (TMLR)*, 2025. Survey Certification.

[23] G. Kudla, A. W. Murray, D. Tollervey, and J. B. Plotkin. Coding-sequence determinants of gene expression in *escherichia coli*. *science*, 324(5924):255–258, 2009.

[24] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.

[25] S. Li, S. Moayedpour, R. Li, M. Bailey, S. Riahi, L. Kogler-Anele, M. Miladi, J. Miner, F. Pertuy, D. Zheng, et al. CodonBERT large language model for mRNA vaccines. *Genome research*, 34(7):1027–1035, 2024.

[26] Y. Li, F. Wang, J. Yang, Z. Han, L. Chen, W. Jiang, H. Zhou, T. Li, Z. Tang, J. Deng, et al. Deep generative optimization of mRNA codon sequences for enhanced protein production and therapeutic efficacy. *bioRxiv*, pages 2024–09, 2024.

[27] R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker. Viennarna package 2.0. *Algorithms for molecular biology*, 6(1):26, 2011.

[28] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[29] V. P. Mauro and S. A. Chappell. A critical analysis of codon optimization in human therapeutics. *Trends in molecular medicine*, 20(11):604–613, 2014.

[30] G. L. Moore and C. D. Maranas. e codonopt: a systematic computational framework for optimizing codon usage in directed evolution experiments. *Nucleic acids research*, 30(11):2407–2416, 2002.

[31] T. H. Olsen, F. Boyles, and C. M. Deane. Observed antibody space: A diverse database of cleaned, annotated, and translated unpaired and paired antibody sequences. *Protein Science*, 31(1):141–146, 2022.

[32] C. N. Pace, F. Vajdos, L. Fee, G. Grimsley, and T. Gray. How to measure and predict the molar absorption coefficient of a protein. *Protein science*, 4(11):2411–2423, 1995.

[33] A. I. Paremskaia, A. A. Kogan, A. Murashkina, D. A. Naumova, A. Satish, I. S. Abramov, S. G. Feoktistova, O. N. Mityaeva, A. A. Deviatkin, and P. Y. Volchkov. Codon-optimization in gene therapy: promises, prospects and challenges. *Frontiers in Bioengineering and Biotechnology*, 12:1371596, 2024.

[34] M. Prakash, A. Moskalev, P. D. Jr., S. Combs, T. Mansi, J. Scheer, and R. Liao. Bridging biomolecular modalities for knowledge transfer in bio-language models. In *Neurips 2024 Workshop Foundation Models for Science: Progress, Opportunities, and Challenges*, 2024.

[35] P. Puigbo, E. Guzmán, A. Romeu, and S. Garcia-Vallvé. OPTIMIZER: a web server for optimizing the codon usage of DNA sequences. *Nucleic acids research*, 35(suppl_2):W126–W131, 2007.

[36] P. M. Sharp and W.-H. Li. The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications. *Nucleic acids research*, 15(3):1281–1295, 1987.

[37] S. Stanton, W. Maddox, N. Gruver, P. Maffettone, E. Delaney, P. Greenside, and A. G. Wilson. Accelerating bayesian optimization for biological sequence design with denoising autoencoders. In *International conference on machine learning*, pages 20459–20478. PMLR, 2022.

[38] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[39] B. Trabucco, X. Geng, A. Kumar, and S. Levine. Design-bench: Benchmarks for data-driven offline model-based optimization. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, pages 21658–21676, 2022.

[40] B. Trabucco, A. Kumar, X. Geng, and S. Levine. Conservative objective models for effective offline model-based optimization. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[41] A. Tripp, E. Daxberger, and J. M. Hernández-Lobato. Sample-efficient optimization in the latent space of deep generative models via weighted retraining. *Advances in Neural Information Processing Systems*, 33:11259–11272, 2020.

[42] R. Turner, D. Eriksson, M. McCourt, J. Kiili, E. Laaksonen, Z. Xu, and I. Guyon. Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. In *NeurIPS 2020 competition and demonstration track*, pages 3–26. PMLR, 2021.

[43] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024.

[44] K. K. Yang, Z. Wu, and F. H. Arnold. Machine-learning-guided directed evolution for protein engineering. *Nature methods*, 16(8):687–694, 2019.

[45] M. Yazdani-Jahromi, M. Prakash, T. Mansi, A. Moskalev, and R. Liao. HELM: Hierarchical encoding for mRNA language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025.

[46] D. Zhang, J. Fu, Y. Bengio, and A. Courville. Unifying likelihood-free inference with black-box optimization and beyond. *arXiv preprint arXiv:2110.03372*, 2021.

[47] H. Zhang, H. Liu, Y. Xu, H. Huang, Y. Liu, J. Wang, Y. Qin, H. Wang, L. Ma, Z. Xun, et al. Deep generative models design mRNA sequences with enhanced translational capacity and stability. *Science*, 390(6773):eadr8470, 2025.

[48] H. Zhang, L. Zhang, A. Lin, C. Xu, Z. Li, K. Liu, B. Liu, X. Ma, F. Zhao, H. Jiang, et al. Algorithm for optimized mRNA design improves stability and immunogenicity. *Nature*, 621(7978):396–403, 2023.

A Extended Related Work

Codon Optimization Codon optimization is a longstanding challenge in synthetic biology and biotechnology, aiming to boost recombinant protein expression by tailoring gene sequences to a host's codon preferences. A visual depiction of the codon optimization problem is provided in Figure 3. Early strategies focused heavily on maximizing the use of highly frequent codons within the host organism, leading to tools like JCat [16] and OPTIMIZER [35] that emphasize codon adaptation index (CAI) and codon pair bias as key metrics. However, such traditional approaches have shown limitations, including insufficient attention to other genomic features such as mRNA stability and resulting protein expression [48, 11, 33].

Recent advances have seen the incorporation of deep learning into the codon optimization pipeline. Pretrained mRNA language models based on transformer architectures, such as CodonBERT [25], CodonTransformer [13] and GEMORNA [47], leverage large-scale, multi-species datasets to learn complex codon usage patterns across diverse organisms via amino-acid–codon pair representations and novel tokenization, ensuring species specificity and preservation of regulatory elements. Similarly, RNop [15] employs a suite of custom loss functions that target fidelity, tRNA availability, mRNA secondary structure, and codon adaptation within a transformer architecture. Other recent work further leverages recurrent networks and active learning to improve translational efficiency across expression systems [20, 26]. These advances achieve superior computational efficiency, fidelity, and improved protein expression compared to earlier methods. Comparative analyses have also highlighted the importance of integrating multiple metrics in the objective, such as CAI, GC content, mRNA folding energy, and codon pair bias, rather than relying on single-metric approaches [48].

Discrete Sequence Optimization Methods for optimizing discrete sequences vary along two axes. The first is the *search space geometry*: whether the optimizer works directly in sequence space, or in a learned continuous latent space that is decoded back into sequences. The second is the *trust mechanism*: what stops the optimizer from exploiting the surrogate model where it is unreliable, i.e., on out-of-distribution designs (e.g., via a conservative surrogate, posterior uncertainty, etc.). We organize prior work by search space geometry, and comment on the trust mechanism in each case.

Sequence-Space Search The first geometry operates directly on sequences (or distributions over them). Population-based methods evolve candidates through mutation and selection: Genetic Algorithms (GA) [10] are simple but sample-inefficient when evaluations are costly [42], and model-based variants add a learned model to screen proposals [44] or guide mutation [46]. Estimation-of-distribution methods instead maintain a learned distribution $q_\theta(x)$ over sequences, repeatedly sampling, reweighting toward high fitness, and refitting [5, 6]. The VAE in [6] acts only as a density *prior* to keep sampling on the same manifold, so the search still lives in sequence space. A second line of work trains a fitness surrogate and runs gradient ascent on a continuous relaxation of the sequence. This approach can fail miserably as the surrogate tends to over-predict on the very inputs the optimizer discovers, since they drift off the training distribution: a failure mode known as reward hacking [40, 39, 22]. The proposed remedies adjust the *trust mechanism* rather than the geometry: conservative objective models regularize the surrogate to under-predict off-distribution [40]; Reinforcement Learning methods generate tokens under a surrogate reward with per-round model selection [2]; and Bayesian optimization over string kernels or embeddings uses posterior uncertainty and trust regions [4].

Latent-Space Optimization The second geometry learns a continuous latent space, shared by a decoder that reconstructs sequences and a model that predicts their properties, and optimizes in that latent space [14, 21, 41]. Latent-Space Bayesian Optimization (LSBO) adds a probabilistic surrogate, generally a Gaussian process or Deep Ensemble, to quantify epistemic uncertainty and balance exploration-exploitation across rounds of propose, test, and update [37, 17, 1]. The upside is that a smooth, low-dimensional space makes gradient-based and Bayesian search easy to apply and additionally acts as a prior that keeps candidates realistic. The downside is the extra decoder, which needs to be trained and can potentially mis-decode, creating the chance for a design to fall off-distribution. 'Reward hacking' is therefore not unique to sequence-space search, and motivates the use of uncertainty safeguards.

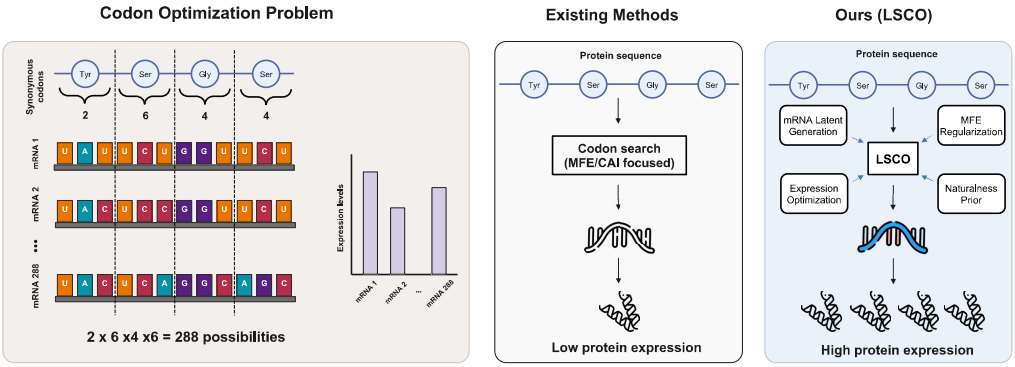


Figure 3: **Generative Codon optimization problem.** Starting from a protein sequence, we navigate the combinatorial codon design space by mapping to and optimizing in a continuous latent space. LSCO integrates differentiable predictors for expression and stability, a naturalness prior, and constrained decoding to produce host-compatible, high-expression mRNA sequences.

Our approach LSCO belongs to the latent-space family but differs in three ways. First, we optimize for *predicted expression* directly, not proxy metrics, within a multi-criteria objective that also weighs mRNA stability and naturalness while maintaining target protein fidelity. Second, rather than training a task-specific autoencoder, we pre-train and reuse an mRNA foundation model and conduct search in its latent space, inheriting realistic codon-usage priors. Third, our expression surrogate model is trained as a deep ensemble, whose model/epistemic uncertainty steers the search away from unreliable regions.

B Implementation details

Back-translation model The back-translation model uses the standard BERT architecture with absolute position embeddings, configured with a hidden dimension of 777, 9 transformer layers, 21 attention heads per layer, a feed-forward dimension of 777, and a maximum sequence length of 2048 tokens. It operates over a joint vocabulary of 91 tokens: 27 amino-acid tokens (20 canonical amino acids plus 7 special tokens `[BOS]`, `[EOS]`, `[CLS]`, `[SEP]`, `[MASK]`, `[PAD]`, and `[UNK]`) as the input alphabet, and 64 codon tokens (one per sense codon) as the output alphabet. During training, amino-acid input tokens are masked with probability $p = 0.2$.

Variational mRNA language model We use a 12-layer ModernBERT architecture [43] with Rotary Position Embeddings (RoPE) [38]. The model uses a vocabulary of 70 tokens (64 codons plus 6 special tokens `[BOS]`, `[EOS]`, `[CLS]`, `[SEP]`, `[MASK]`, and `[PAD]`), a hidden dimension of 768, 12 attention heads per layer, and a feed-forward dimension of 3072. The VMAE masking ratio is set to $p = 0.15$. The model is split at layer $k = 6$, using the first six layers as the encoder E and the remaining six as the decoder D. We use a standard Gaussian prior $\mathcal{N}(0, I)$ over the latent variables $\mathbf{z} \in \mathcal{Z}_L$. Pretraining is performed for 40 epochs using AdamW [28] with a batch size of 256, an initial learning rate of 10^{-4}, linear warmup over the first 10% of training steps, and cosine learning-rate decay thereafter.

Expression predictor We implement Ψ as a dual-chain transformer that models heavy and light chain mRNA separately, then aggregates the two chain representations using Multiple Instance Learning (MIL) attention before a final linear layer predicts antibody expression. In practice, we found that reusing the pretrained language model encoder E as the backbone for Ψ improves the expression prediction. To enable this, we replace the first `nn.Embedding` layer of E with a dense `nn.Linear` layer to handle soft non-one-hot sequences. Once trained, Ψ is differentiable with respect to the soft-sequence inputs $\tilde{\mathbf{r}}_\tau$, allowing gradient-based optimization. Training proceeds for 120 epochs with Negative Log-Likelihood loss (for each member m of the ensemble) on the mAb expression (absorbance measured at 280 nm) data [34, 45]. We use AdamW optimization with a batch size of 64, initial learning rate of 5×10^{-5}, and the same warmup and decay schedule as the

Algorithm 1 Latent-Space Codon Optimizer (LSCO)

Require: Protein sequence $\mathbf{a}$, pretrained encoder E, decoder D, predictors Ψ and Υ, weights $\lambda_\Psi, \lambda_\Upsilon, \lambda_{\text{nat}}$, initial τ_{high}, final τ_{low}, optimization steps T, learning rate η
Ensure: Optimized mRNA sequence $\mathbf{c}^\star$

```
Initialize z ∈ Z_L                                   ▷ randomly from N(0, I)
for t = 1 to T do
    τ ← τ_high + (t/T)(τ_low − τ_high)                ▷ temperature annealing
    r̃_τ ← softmax_τ(D(z) + M(a))                     ▷ constrained soft sequence
    L ← λ_Ψ U_Ψ(r̃_τ) − λ_MFE Υ(r̃_τ) + λ_nat U_nat(z, z_nat)
    z ← z + η ∇_z L                                  ▷ gradient ascent on the objective
end for
r ← arg max(D(z) + M(a))                             ▷ discrete one-hot mRNA
c* ← map r to codon sequence
return c*
```

language model. The Spearman's rank correlation between the predicted and ground truth expression on the test split reaches 0.65.

We operate Ψ on the relaxed sequence $\tilde{\mathbf{r}}_\tau$ rather than the latent space $\mathcal{Z}_L$ for two reasons: (i) early experiments showed that optimizing directly in $\mathcal{Z}_L$ can cause large shifts in predicted expression and latent values without corresponding changes at the sequence level; and (ii) the softmax representation admits the simple constrained-decoding step used to enforce $\tilde{\mathbf{r}}_\tau \in \mathcal{S}(\mathbf{a})$.

MFE predictor Analogously, the MFE predictor Υ mirrors Ψ's architecture. It is trained for 50 epochs using MSE loss on 50,000 sub-sampled OAS mRNA sequences, with ground-truth MFE labels computed using the ViennaRNA package. Optimization follows identical settings to Ψ. The Spearman's rank correlation between the predicted and ground truth MFE on the test split reaches 0.91.

Constrained Decoding Both the back-translation model and the optimizer require that every decoded sequence translates exactly to the target protein $\mathbf{a}$, enforced with a shared masking mechanism. Let $\mathbf{l} = D'(\mathbf{z}) \in \mathbb{R}^{L\times 64}$ be the pre-softmax decoder logits. For each position a_j we define a validity mask

$$M(a_j)[i] = \begin{cases} 0, & \text{if codon } i \in \mathcal{C}(a_j), \\ -\infty, & \text{otherwise,} \end{cases} \tag{4}$$

and compute masked logits $l_{j,i} \leftarrow l_{j,i} + M(a_j)[i]$, then apply the temperature-τ softmax $\tilde{\mathbf{r}}_\tau = \text{softmax}_\tau(\mathbf{l})$. This guarantees $\tilde{\mathbf{r}}_\tau \in \mathcal{S}(\mathbf{a})$ while allowing free exploration of synonymous codons at each position.

LSCO hyperparameters We determined the weights λ_Ψ and λ_Υ via grid search on each dataset to maximize predicted A280 while ensuring the optimized sequence's MFE is generally lower than the wild-type/original MFE, with flexibility to relax this constraint when predicted A280 ≤ 1. In practice, LSCO proved robust to a wide range of these weights. Grid search identified $\lambda_\Psi = 10$ and $\lambda_\Upsilon = 0.1$ as effective. For decoding, we set the initial temperature $\tau_{\text{high}} = 3$ and the final temperature $\tau_{\text{low}} = 0.01$. These temperature parameters showed higher sensitivity, particularly τ_{low}; excessively high values caused significant rounding errors, reducing expression, while overly low values hindered optimization progress. We ran LSCO for $T = 3000$ iterations, observing that additional iterations improved convergence marginally. The learning rate was set to $\eta = 10^{-4}$, and, leveraging the differentiability of the LSCO objective, we employed the Adam optimizer for robust gradient-based updates.

Decoding Temperature In practice, we also observed the importance of carefully selecting the decoding temperature τ. Too high temperature allows for broader exploration of the synonymous sequence space but can introduce significant rounding errors when mapping $\tilde{\mathbf{r}}_\tau$ to the nearest discrete mRNA sequence $\mathbf{r}$ via argmax. Conversely, too low temperature hinders exploration of the soft-sequence space by creating high-energy barriers between sequences. Empirically, we found that linearly annealing τ from a high to a low value during optimization strikes the best balance. This

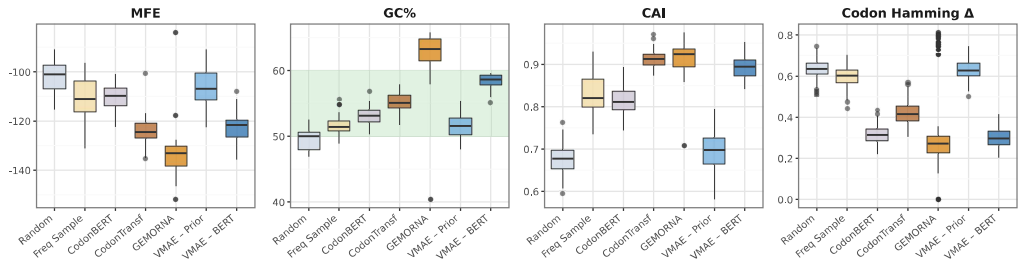


Figure 4: **Sequence-level biophysical proxies across generation methods.** Minimum Free Energy (MFE; lower = more stable), GC Content (GC%; shaded band marks host-appropriate range), Codon Adaptation Index (CAI; higher is better), and codon Hamming distance (Δ) from the reference sequence (lower = fewer codon edits), for non-contextual baselines (*Random*, *Freq Sample*), modern learned generators (*CodonBERT*, *CodonTransf*, *GEMORNA*), and our pretrained models (*VMAE–Prior*, decoding from Gaussian latent prior; *VMAE–BERT*, decoding from CodonBERT generated sequence's latent space).

strategy allows the optimizer to first explore the domain freely to identify promising candidates and then commit to a single solution at the end of the process

C Additional Results and Experiments

C.1 Pretrained mRNA language models comparison

To evaluate whether pretrained mRNA language models learn biologically meaningful codon patterns, we compared four sequence generation strategies: random codon assignment (*Random*), codon sampling according to empirical codon frequencies (*Freq Sample*), an amino-acid-to-codon translation model (*CodonBERT*) pretrained on OAS sequences, and a codon-to-codon Variational Masked Auto-Encoder (*VMAE*) also pretrained on the OAS data. For VMAE generation, we include two variants, one that samples new sequences starting from the prior latent space (*VMAE – Prior*) and one that samples sequences starting from the encoded CodonBERT generated sequences (*VMAE – BERT*). All sequences are evaluated on Minimum Free Energy (MFE), GC content, Codon Adaptation Index (CAI), and codon-level Hamming distance from the reference starting sequence.

As shown in Figure 4, modern pretrained mRNA language models generate sequences with substantially improved biological properties than other baselines. CodonTransf, GEMORNA and VMAE – BERT all score favorably in terms of lower MFE, higher GC content (with GEMORNA falling just above the ideal host-specific GC range), higher CAI values, and lower codon Hamming distances. This indicates that modern transformer-based pretrained models efficiently learn context-dependent synonymous codon substitutions that improve quality without unnecessary sequence changes.

C.2 LSCO Optimization Trajectory

Figure 5 illustrates how LSCO reshapes the distribution of sequences in the learned latent space. The starting sequences (red triangles) reside in a cluster (C7) with low averaged predicted expression ($\bar{a} = 1.808$), whereas the LSCO-optimized sequences (gold stars) are consistently relocated toward clusters with substantially higher averaged predicted A280 ($\bar{a} \approx 3.0$–3.4). All optimization trajectories follow a similar direction, indicating that LSCO discovers coherent ascent paths on the underlying fitness landscape rather than arbitrary sequence perturbations. The companion uncertainty panel (right) shows that the optimized sequences stay in regions of low epistemic uncertainty, confirming that the LCB utility keeps the search in-distribution rather than exploiting the predictor off-manifold.

C.3 Codon-Usage Analysis

To interpret the strategy learned by *LSCO – LCB + MFE*, Figure 6 shows codon-level transition probabilities between original and optimized sequences, broken down by amino acid, over the 54 selected mAbs, i.e. which synonymous codons LSCO tends to replace and which it preserves. Several

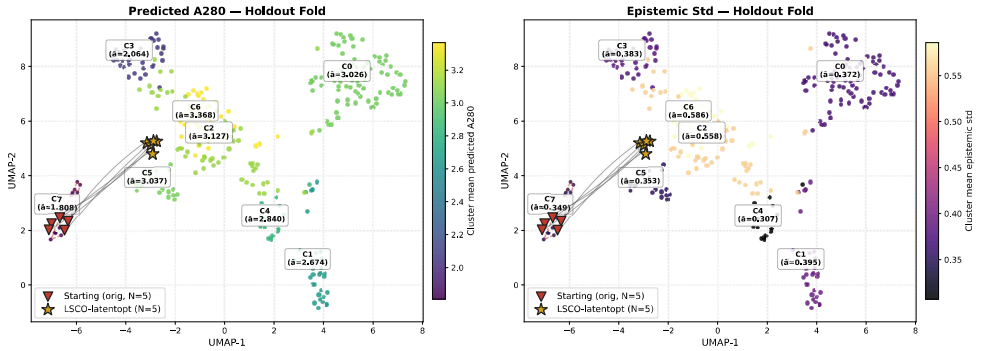


Figure 5: **LSCO optimization trajectories in latent space.** UMAP projection of the pretrained VMAE latent space on a holdout fold, coloured by predicted A280 (left) and by predictor epistemic uncertainty (right). Starting sequences (red triangles) lie in a low-expression cluster (C7, $\bar{a} = 1.808$); LSCO-optimized sequences (gold stars) are relocated toward high-expression clusters ($\bar{a} \approx 3.0$–3.4). Trajectories (grey arrows) follow a consistent direction and remain in low-uncertainty regions.

amino-acid families exhibit strong off-diagonal structure, indicating that optimization consistently redirects codon usage toward a preferred subset of alternatives rather than making random synonymous substitutions. There is also a moderate shift toward GC-richer codons across multiple families, consistent with the GC-content and MFE improvements of *LSCO – LCB + MFE* in Figure 2. However, the transition patterns are highly amino-acid-specific, suggesting a more nuanced strategy than simple GC maximization; the Symmetrized KL values confirm that the magnitude of redistribution varies substantially across families, as LSCO concentrates its modifications on specific synonymous groups.

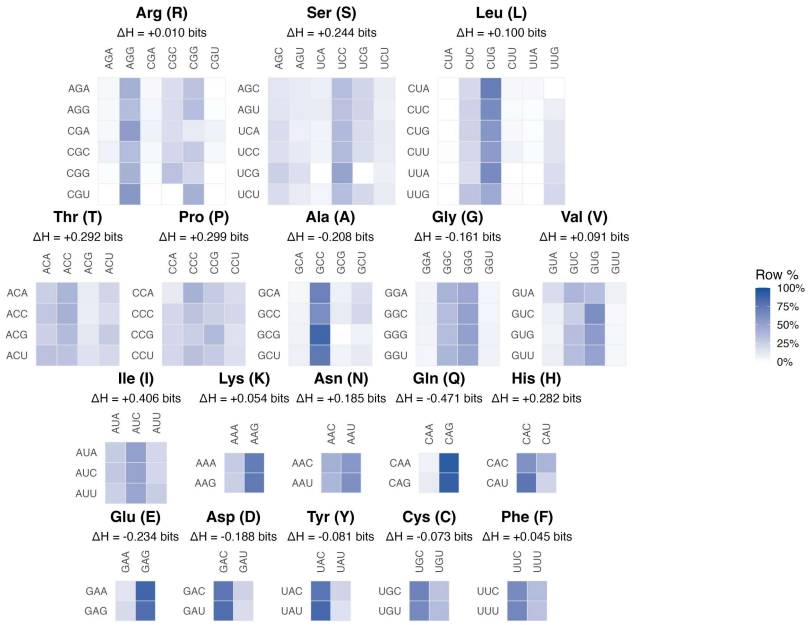
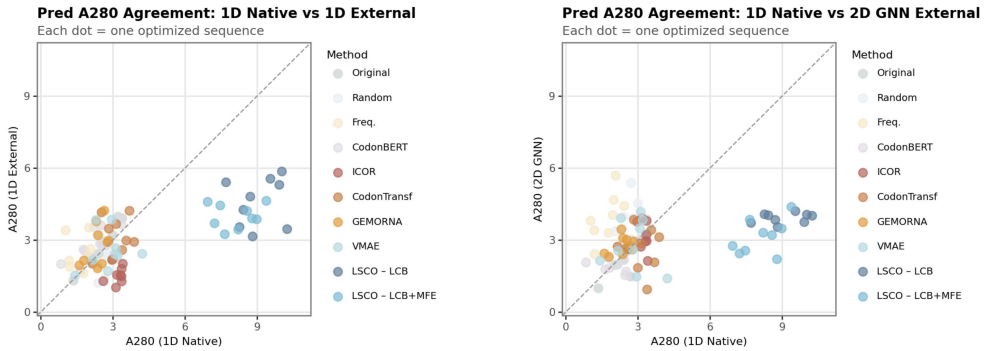


Figure 6: **Codon substitutions induced by LSCO optimization.** Rows denote original codons, columns optimized codons; colours show row-normalized transition probabilities for synonymous substitutions produced by *LSCO – LCB + MFE*, over the 54 starting mAbs. Strong diagonal structure indicates codon conservation, while off-diagonal transitions reveal learned preferences. The Symmetrized KL (SKL) divergence quantifies the magnitude of codon redistribution per amino-acid family (higher SKL $\rightarrow$ larger redistribution).



(a) Native 1D vs External 1D

(b) Native 1D vs External 2D GNN.

Figure 7: **Predicted-A280 agreement across evaluators.** Each dot is one optimized sequence, coloured by method; the dashed line marks perfect agreement. The horizontal axis is the native 1D predictor Ψ; the vertical axis is an independent external predictor withheld from optimization, the 1D Shallow-AE model (a) and the structure-aware 2D GNN (b). LSCO variants (blue) are rated highest by the native model, but score favorably also under the two external models in terms of ordering/ranking preserving.

C.4 A280 Predictors and Agreement

1D Sequence-Based Predictors. Both 1D predictors follow the same dual-chain design: heavy and light chain mRNAs are encoded separately, each chain is reduced to a fixed-size representation, and the two are aggregated with a Multiple Instance Learning (MIL) attention module before a final linear layer predicts A280. The two variants then differ mainly in the encoder backbone. The *native* predictor (Ψ) reuses the pretrained VMAE encoder E as its backbone (with the first embedding layer replaced by a dense layer so it accepts the relaxed, soft-sequence input $\tilde{\mathbf{r}}_\tau$); this is the model whose gradients drive LSCO. The *external* 1D predictor keeps similar dual-chain + MIL head but swaps in a different, shallower autoencoder backbone that never takes part in optimization. It therefore provides an independent read-out that shares the 1D sequence view but not the representation LSCO is optimized against.

2D Structure-Based Predictor. The external "2D GNN" predictor takes a genuinely different view of each sequence. For each chain, the mRNA is folded and its dot-bracket secondary structure is converted into a graph, with one-hot nucleotide node features and edges from both the backbone and the predicted base pairs. A graph neural network encodes this structure, and the same MIL-attention aggregation combines the two chains before the regression head. Because it consumes secondary-structure graphs rather than raw sequence embeddings, the 2D GNN carries a structural inductive bias absent from both 1D models, making it the most independent of the three evaluators.

Cross-predictor agreement. Figure 7 plots, for every optimized sequence, the native 1D A280 against each external predictor. In both panels the LSCO points (blue) sit clearly at the center-right, separated from all baselines: this implies they are rated highest by the native predictor and rated comparably or slightly higher by the external 1D and the structure-aware 2D GNN. The external predictors are more conservative in absolute terms in that most points lie below the diagonal, so agreement is more on *ranking* rather than exact value prediction. Overall ordering is more or less preserved across all three, despite distinct structural views. This consistency across predictors with different backbones and inductive biases presents some evidence that the gains are not an artifact of Ψ being exploited off-distribution, but reflect genuinely higher predicted expression.

C.5 Deep-Ensemble and Epistemic Uncertainty Aware Objective

Probabilistic Deep Ensemble. We implement the expression predictor Ψ as a deep ensemble of $M = 10$ *probabilistic* members [24]. Each member m does not output a point estimate but a full Gaussian predictive distribution over A280:

$$p_m(y \mid \tilde{\mathbf{r}}_\tau) = \mathcal{N}\big(y;\, \mu_m(\tilde{\mathbf{r}}_\tau),\, \sigma_m^2(\tilde{\mathbf{r}}_\tau)\big)\,, \tag{5}$$

		Predicted A280			Metrics (VH + VL)		
λ_{MFE}	λ_{LCB}	1D Transf (Internal)	1D Transf (External)	2D GNN	MFE	GC%	CAI
0	0	$\mathbf{9.585 \pm 0.218}$	$\mathbf{4.518 \pm 0.223}$	$\mathbf{3.492 \pm 0.203}$	-235.187 ± 3.555	53.303 ± 0.423	0.816 ± 0.008
	1	9.156 ± 0.241	4.400 ± 0.254	3.485 ± 0.212	-236.711 ± 3.603	53.601 ± 0.390	0.815 ± 0.009
0.1	0	8.180 ± 0.237	3.900 ± 0.224	3.175 ± 0.217	-266.274 ± 2.783	57.756 ± 0.285	0.830 ± 0.009
	1	7.858 ± 0.235	3.935 ± 0.265	3.099 ± 0.187	-267.050 ± 3.211	57.916 ± 0.300	0.837 ± 0.008
1	0	8.065 ± 0.232	3.887 ± 0.200	3.319 ± 0.201	-263.765 ± 3.122	58.003 ± 0.304	0.825 ± 0.009
	1	7.899 ± 0.256	3.951 ± 0.271	2.923 ± 0.182	-266.365 ± 3.381	58.018 ± 0.317	$\mathbf{0.838 \pm 0.008}$
5	0	7.921 ± 0.260	3.729 ± 0.220	3.057 ± 0.220	-265.006 ± 3.443	57.959 ± 0.307	0.824 ± 0.009
	1	7.635 ± 0.271	3.852 ± 0.228	3.161 ± 0.249	-263.954 ± 3.377	57.973 ± 0.321	0.832 ± 0.008
10	0	7.490 ± 0.248	3.514 ± 0.263	3.089 ± 0.248	$\mathbf{-267.381 \pm 3.033}$	58.494 ± 0.282	0.818 ± 0.008
	1	7.390 ± 0.264	3.500 ± 0.256	3.169 ± 0.241	-265.220 ± 2.735	58.233 ± 0.258	0.829 ± 0.009

Table 2: **MFE sensitivity analysis for LSCO variants.** Predicted A280 values from three surrogate models (1D Transformer (Internal), 1D Transformer (External), 2D GNN) and biophysical metrics (minimum free energy, GC content, codon adaptation index) reported for the joint heavy + light chain construct on the 54 starting sequences. The second row ($\lambda_{MFE} = 0, \lambda_{LCB} = 1$) is the LCB-only LSCO baseline ('LSCO – LCB' in Figure 2). The fourth row ($\lambda_{MFE} = 0.1, \lambda_{LCB} = 1$) is the LCB+MFE baseline ('LSCO – LCB + MFE' in Figure 2) that best balances A280 and MFE objectives. **Bold** indicates the best value per column: highest for A280 predictions and CAI, most negative for MFE. GC% is not bolded as "best" is target-dependent. Values are mean ± 95% confidence bands across starting sequences.

where the head outputs both a mean μ_m and a variance σ_m^2. Members are trained *independently*, each with its own random initialization, optimizer, and backward pass, by minimizing the Gaussian negative log-likelihood loss:

$$\mathcal{L}_{\text{NLL}}^{(m)} = \tfrac{1}{2} \log \sigma_m^2(\tilde{\mathbf{r}}_\tau) + \frac{\left(y - \mu_m(\tilde{\mathbf{r}}_\tau)\right)^2}{2\,\sigma_m^2(\tilde{\mathbf{r}}_\tau)}, \tag{6}$$

so that the predicted variance is calibrated to each member's own residuals. The variance head itself is configurable (constant, scalar, per-group, or fully heteroscedastic); we use the scalar σ_m^2 setting, as we noticed data exhibit somehow uniform noise.

Aleatoric/Epistemic Uncertainty Decomposition. Treating the ensemble as a uniformly-weighted Gaussian mixture, the ensemble mean and total predictive variance are

$$\mu_\Psi(\tilde{\mathbf{r}}_\tau) = \frac{1}{M} \sum_{m=1}^{M} \mu_m(\tilde{\mathbf{r}}_\tau), \tag{7}$$

$$\sigma_\Psi^2(\tilde{\mathbf{r}}_\tau) = \underbrace{\frac{1}{M} \sum_{m=1}^{M} \sigma_m^2(\tilde{\mathbf{r}}_\tau)}_{\text{aleatoric}} + \underbrace{\frac{1}{M} \sum_{m=1}^{M} \left(\mu_m(\tilde{\mathbf{r}}_\tau) - \mu_\Psi(\tilde{\mathbf{r}}_\tau)\right)^2}_{\text{epistemic}}. \tag{8}$$

The two terms have distinct meanings. The *aleatoric* term is the average of the members' own predicted variances and captures irreducible noise in the measurement/data. The *epistemic* term is the variance *across* the members' means and captures model uncertainty: it is large where the members disagree, which happens precisely in regions of sequence space that are under-represented in the training data. Because epistemic uncertainty shrinks as designs stay close to the training distribution and grows off-distribution, it is exactly the signal the LCB utility $U_\Psi = \mu_\Psi - \lambda_{\text{LCB}}\,\sigma_\Psi$ (Eq. 3) exploits: penalizing σ_Ψ steers the optimizer away from over-optimistic, out-of-distribution sequences. The same probabilistic ensemble construction is used for all three predictors (1D native, 1D external, 2D GNN); the 2D GNN members simply own an independent structure-graph backbone each.

Epistemic uncertainty of the optimized sequences. Figure 8 reports the epistemic standard deviation ($\sqrt{\text{epistemic}}$) of each predictor on the same 9 optimized mAbs as Figure 2. Two patterns stand out. First, under the *native* predictor (left), the LSCO variants sit at low and tightly-concentrated epistemic uncertainty, comparable to the unoptimized VMAE sequences and well below frequency-based, ICOR, CodonTransformer, and GEMORNA. This confirms that the LCB penalty keeps the search in-distribution rather than chasing high-variance designs. Second, under the two *external*

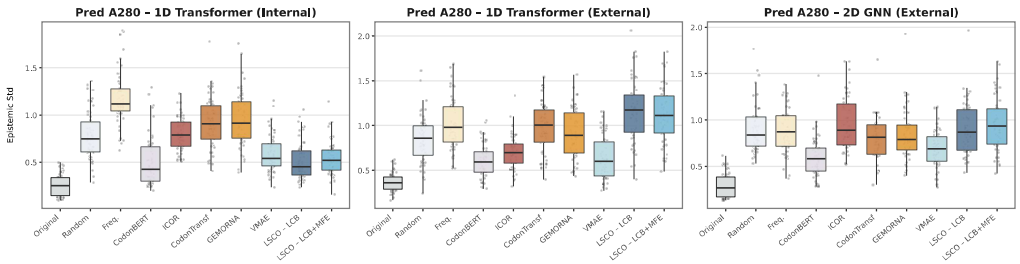


Figure 8: **Epistemic uncertainty of the optimized sequences.** Epistemic standard deviation (across-member disagreement, $\sqrt{\text{epistemic}}$) of the three probabilistic deep-ensemble predictors — 1D Transformer Internal/Native (left), 1D Transformer External (middle), 2D GNN External (right) — evaluated on the same 9 optimized mAbs as Figure 2; each dot is one mAb. Under the native predictor, LSCO variants (blue) show low, concentrated epistemic uncertainty, comparable to the unoptimized VMAE and well below the other optimizers, confirming the LCB utility keeps the search in-distribution. Under the external predictors — never seen during optimization — LSCO's uncertainty is comparable to other high-expression methods, as expected for held-out evaluators.

predictors (middle, right), LSCO's epistemic uncertainty rises to a level similar to the other high-expression methods. This is because the external models were never optimized against, so any strong-expression design is somewhat novel to them, and LSCO is no more off-distribution for them than the competing optimizers. Read together with Figure 2, LSCO therefore attains the *highest* predicted A280 across all three predictors while keeping the *lowest* epistemic uncertainty under the predictor that actually guides optimization, i.e., its gains do not come from exploiting high-uncertainty regions of its own surrogate.

C.6 MFE regularization sensitivity

Table 2 reports the full sweep over the MFE weight λ_{MFE} and the uncertainty-penalty toggle λ_{LCB}, extending the two settings shown in the main text (Figure 2). Increasing λ_{MFE} from 0 drives MFE substantially lower and lifts GC and CAI into the host-appropriate range, at a modest cost in predicted A280; gains in stability saturate beyond $\lambda_{\text{MFE}} = 0.1$, confirming it as the sweet spot used throughout.

C.7 Seed sensitivity study

To assess how much results depend on the random seed for *stochastic* codon optimization methods, we fix a single representative starting mAb (chosen near the median A280 of the dataset) and re-run each stochastic method across 10 seeds. Since the evaluators are deterministic given a sequence, the spread in Figure 9 reflects methods' stochasticity alone. Deterministic methods (original, natural, frequency, ICOR) are seed-inert and therefore omitted from the boxes. We report both LSCO variants (*LSCO – LCB*, *LSCO – LCB + MFE*) alongside the stochastic baselines CodonTransformer (temperature 0.4) and GEMORNA, plus the LSCO random-substitution baseline.

Across seeds, the results in terms of performance generally hold with some caveats. Under the two 1D predictors, both LSCO variants stay above the stochastic baselines on predicted A280 for every seed, with bounded spread; *LCB* variant is tightest, while *LCB + MFE* shows slightly more variability and slightly lower absolute A280. Under the more conservative and less aligned 2D GNN, the boxes overlap more and seed spread is larger for all methods. Biophysical metrics are stable: both LSCO variants keep GC within the 50–60% band on every seed, and *LCB + MFE* consistently yields lower MFE than *LCB*. These results are limited to a single starting mAb and characterize generator stochasticity rather than variation across antibodies.

D LSCO vs. Direct Input-Space Optimization (DISO)

DISO is an ablation of LSCO designed to isolate the contribution of the pretrained VMAE mRNA language model. The two methods share everything except *where* the optimization happens: DISO reuses the exact same expression-predictor ensemble Ψ, MFE head Υ, tokenizers, valid-codon mask,

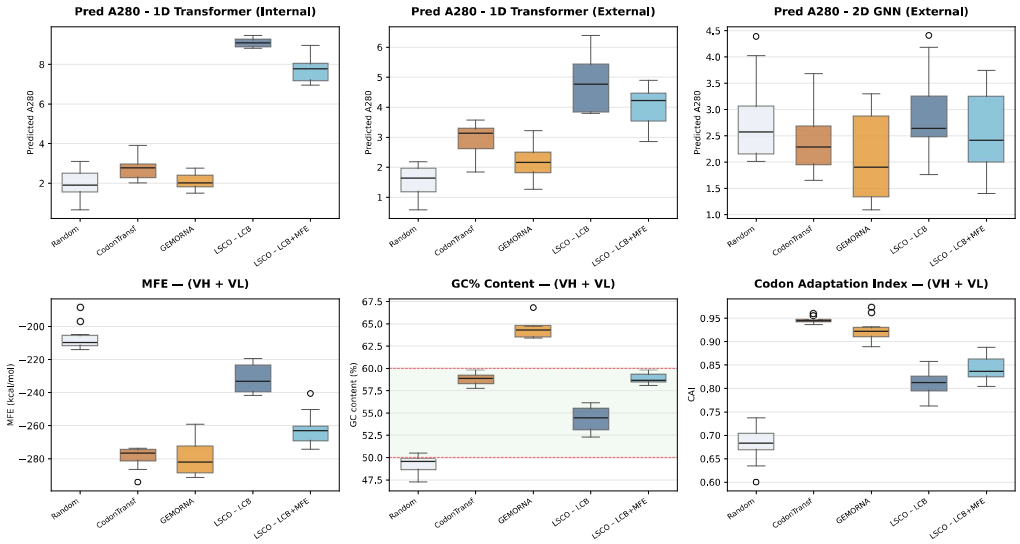


Figure 9: **Seed sensitivity on a single mAb (10 seeds).** Boxes show the spread over different 10 seeded runs, reflecting the stochasticity behind different codon optimization methods. *Top:* predicted A280 from the native 1D transformer (internal), an external 1D transformer, and an external 2D GNN. *Bottom:* biophysical metrics for the joint heavy+light construct (VH+VL): MFE (lower = more stable), GC% (shaded band = host-appropriate 50–60%), and CAI. Both LSCO variants remain above the stochastic baselines on the 1D predictors across all seeds with bounded spread, keep GC within the admissible band, and *LSCO – LCB + MFE* consistently lowers MFE relative to *LSCO – LCB*; seed spread is largest under the more conservative 2D GNN evaluator.

and candidate-selection gate as LSCO. The only thing that changes is the set of variables being optimized, and, as a direct consequence, how the naturalness prior is expressed. The empirical outcome of this comparison is reported in the main text (Table 1): LSCO generally matches or exceeds DISO across evaluators, confirming that the pretrained latent space, and not merely the shared components, drives the gains.

What is being optimized (search geometry). LSCO optimizes a continuous latent code z in the VMAE latent space, and turns it into a codon sequence by passing it through the pretrained decoder D_τ at every step. DISO removes the decoder from the loop entirely: it optimizes one set of *per-position codon logits* directly in sequence space, one logit vector for each position of the heavy and light chains. In other words, LSCO moves through the smooth, learned manifold of the language model, while DISO moves through the raw codon-logit space with no generative model shaping the trajectory. Everything downstream — the temperature-controlled softmax, the valid-codon masking that keeps the sequence translating back to the target protein, the expression and MFE predictors — is identical.

Naturalness prior: latent anchoring vs. distribution matching. This is the component that changes most, and it changes because it *has* to. In LSCO the naturalness prior lives in the latent space: we encode the BERT back-translated "natural" sequence to a reference code z_{nat} and pull the current code toward it with a squared-distance penalty, $U_{\text{nat}}(z) = -\|z - z_{\text{nat}}\|^2$. DISO has no latent space, so this anchor cannot be defined the same way. Instead, DISO anchors the sequence directly in codon space: it builds a fixed per-position categorical distribution from the BERT-natural sequence (using a fixed reference temperature $\tau_{\text{BERT}} = 0.5$) and, at each step, penalizes the Kullback–Leibler divergence between the current per-position codon distribution and this fixed reference, $\mathrm{KL}(p_{\text{current}} \,\|\, p_{\text{BERT}})$. For a two-chain antibody the term is the average of the heavy- and light-chain KL divergences. Conceptually both do the same job — keep candidates close to biologically plausible codon usage — but LSCO measures "closeness" as a distance between latent codes, whereas DISO measures it as a distance between probability distributions over codons. Setting $\lambda_{\text{nat}} = 0$ removes the anchor in both methods.

	LSCO	DISO
Optimized variable	latent code z	per-position codon logits
Generative decoder in loop	Yes (D_τ)	No
Naturalness anchor	latent ℓ^2 distance $\|z - z_{\text{nat}}\|^2$	KL to fixed BERT distribution
Naturalness weight λ_{nat}	10^{-2} (0 to disable)	10^{-2} (0 to disable)
Initialization	$z \sim \mathcal{N}(0, I)$	BERT-natural warm start (or $\mathcal{N}(0, 0.5^2)$)
Temperature schedule τ	$0.5 \to 0.03$	$0.5 \to 0.03$
Optimization steps	3000	3000
Learning rate	5×10^{-3}	5×10^{-3}
MFE weight λ_{MFE}	$\{0, 0.1, 1\}$	$\{0, 0.1, 1\}$
Optimizer	AdamW	AdamW

Table 3: Configuration differences between LSCO and DISO. All model components (expression ensemble, MFE head, tokenizers, valid-codon mask, candidate gate) are shared; only the search geometry and its downstream consequences differ.

Initialization. LSCO initializes the latent code z from a standard Gaussian and lets the decoder map it to a starting sequence. DISO offers two starting points. The default (`init_mode=bert`) warm-starts the logits at the BERT-natural codon for each position (a logit of 0 for the natural codon and $-\delta$ for the others, with $\delta = 3.0$), so DISO begins from a natural sequence. The "fairer-ablation" variant (`init_mode=random_gauss`) instead draws logits from $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.5$ and no bias toward the natural codon, immediately suppressing invalid codons via the valid-codon mask so optimization still starts from a protein-consistent point. Importantly, the naturalness anchor target is *always* the BERT-natural distribution regardless of where the logits are initialized — initialization only decides the starting point, not the anchor.

Optimization schedule and other hyperparameters. To keep the comparison as clean as possible, both methods share the same optimization budget and schedule: an identical decoding-temperature anneal from a high value (broad exploration) down to a low value (commit to a single codon per position), the same ramp on the constraint that enforces protein consistency, the same number of steps, learning rate, optimizer, and range of MFE weights. This is deliberate — because DISO differs from LSCO only in the search geometry (raw codon logits vs. the pretrained latent code) and the induced form of the naturalness anchor, matching all remaining settings ensures that any performance gap is attributable to those two factors rather than to differing training budgets. Table 3 lists the settings for both.